

Microprocessor

- Microprocessor (μ P) is the “**brain**” of a computer that has been **implemented on one semiconductor chip**.
- The word comes from the combination **micro** and **processor**.
- Processor means a device that processes whatever(binary numbers, 0's and 1's)
 - To **process** means to **manipulate**. It describes all manipulation.
 - Micro - > extremely small

Definition of a Microprocessor.

The microprocessor is a programmable device that takes in numbers, performs on them arithmetic or logical operations according to the program stored in memory and then produces other numbers as a result.

Microprocessor ?

*A microprocessor is multi programmable clock driven register based **semiconductor device** that is used to **fetch , process & execute** a data within fraction of seconds.*

Applications

- Calculators
- Accounting system
- Games machine
- Instrumentation
- Traffic light Control
- Multi user, multi-function environments
- Military applications
- Communication systems

Evolution of Microprocessor

First Generation (1939 -1954)

Vacuum Tubes

Second Generation (1954 -1959)

Transistor

Third Generation (1959 -1971)

Integrated Circuit (IC)

Fourth Generation (1971 -Present)

Microprocessor

Microprocessor is scaling from 4004 to Pentium 4.

Microprocessor is identified with word size of data

MICROPROCESSOR HISTORY

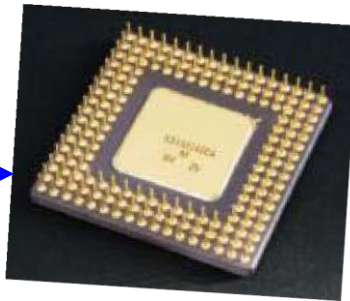
DIFFERENT PROCESSORS AVAILABLE

Socket



Pinless
Processor

Processor



Slot
Processor



Processor Slot

Development of Intel Microprocessors

- **8086 - 1979**
- **286 - 1982**
- **386 - 1985**
- **486 - 1989**
- **Pentium - 1993**
- **Pentium Pro - 1995**
- **Pentium MMX -1997**
- **Pentium II - 1997**
- **Pentium II Celeron - 1998**
- **Pentium II Zeon - 1998**
- **Pentium III - 1999**
- **Pentium III Zeon - 1999**
- **Pentium IV - 2000**
- **Pentium IV Zeon - 2001**

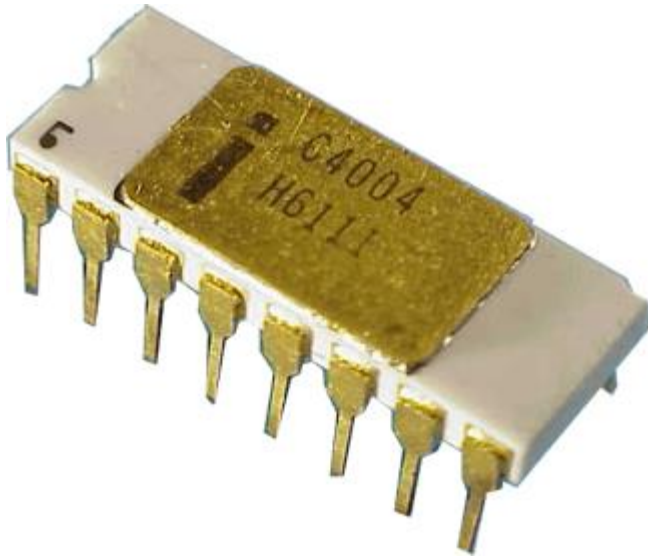
GENERATION OF PROCESSORS

Processor	Bits	Speed
8080	8	2 MHz
8086	16	4.5 – 10 MHz
8088	16	4.5 – 10 MHz
80286	16	10 – 20 MHz
80386	32	20 – 40 MHz
80486	32	40 – 133 MHz

GENERATION OF PROCESSORS

Processor	Bits	Speed
Pentium	32	60 – 233 MHz
Pentium Pro	32	150 – 200 MHz
Pentium II, Celeron , Xeon	32	233 – 450 MHz
Pentium III, Celeron , Xeon	32	450 MHz – 1.4 GHz
Pentium IV, Celeron , Xeon	32	1.3 GHz – 3.8 GHz
Itanium	64	800 MHz – 3.0 GHz

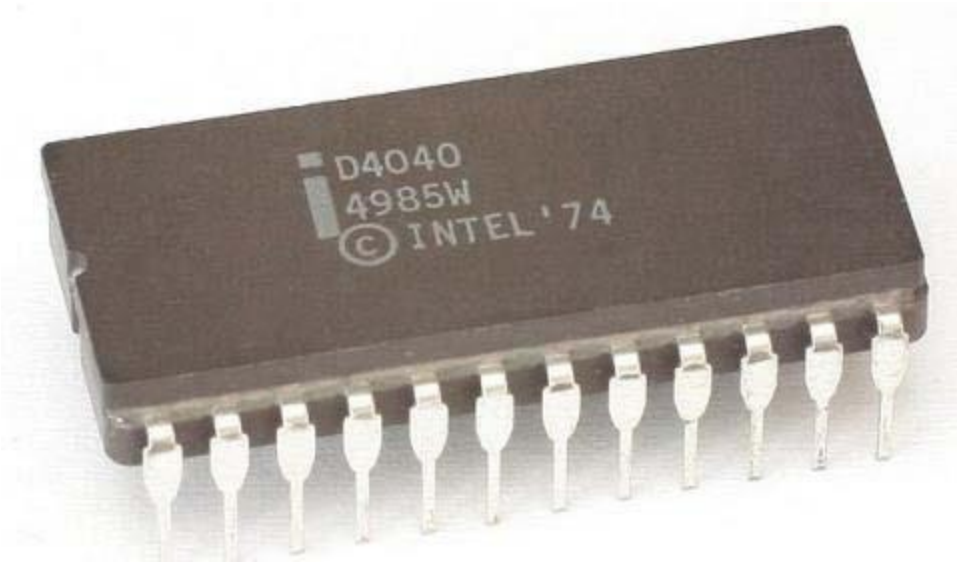
Intel 4004



- Introduced in 1971.
- It was the first microprocessor by Intel.
- It was a 4-bit μ P.
- Its clock speed was 740KHz.
- It had 2,300 transistors.
- It could execute around 60,000 instructions per second.

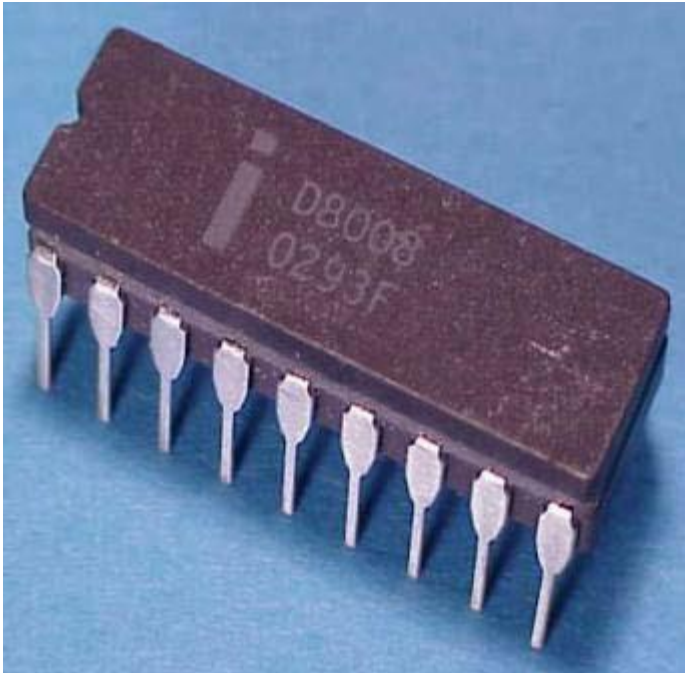
Intel 4040

- Introduced in 1971.
- It was also 4-bit μ P.



8-bit Microprocessors

Intel 8008



- Introduced in 1972.
- It was first 8-bit μ P.
- Its clock speed was 500 KHz.
- Could execute 50,000 instructions per second.

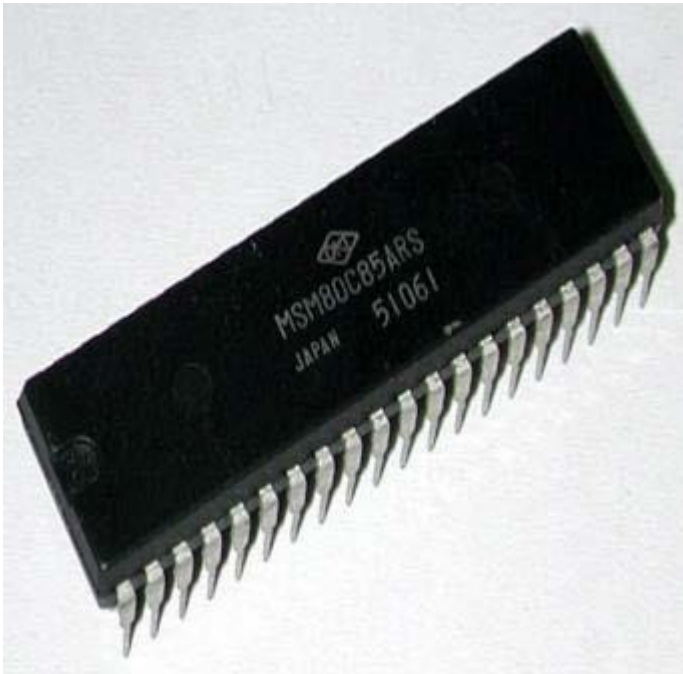
Intel 8080



- Introduced in 1974.
- It was also 8-bit μ P.
- Its clock speed was 2 MHz.
- It had 6,000 transistors.

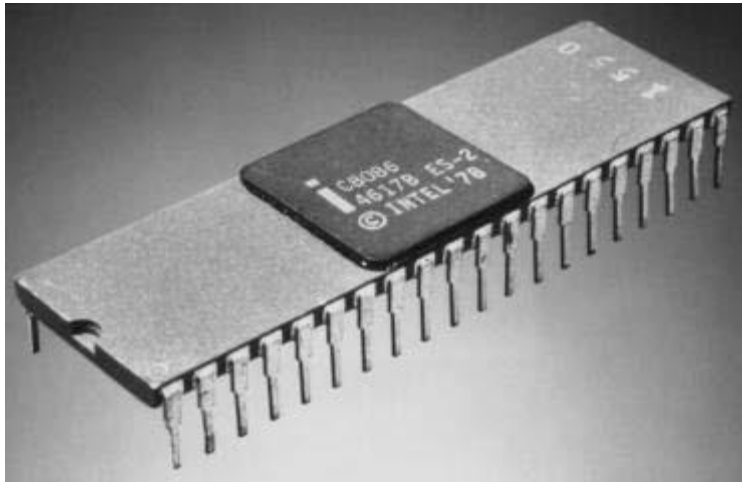
Intel 8085

- Introduced in 1976.
- It was also 8-bit μ P.
- Its clock speed was 3 MHz.
- Its data bus is 8-bit and address bus is 16-bit.
- It had 6,500 transistors.
- Could execute 7,69,230 instructions per second.
- It could access 64 KB of memory.
- It had 246 instructions.



16-bit Microprocessors

INTEL 8086



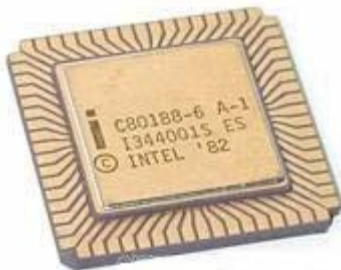
- Introduced in 1978.
- It was first 16-bit μ P.
- Its clock speed is 4.77 MHz, 8 MHz and 10 MHz, depending on the version.
- Its data bus is 16-bit and address bus is 20-bit.
- It had 29,000 transistors.
- Could execute 2.5 million instructions per second.
- It could access 1 MB of memory.
- It had 22,000 instructions.
- It had ***Multiply*** and ***Divide*** instructions.

INTEL 8088



- Introduced in 1979.
- It was also 16-bit μ P.
- It was created as a cheaper version of Intel's 8086.
- It was a 16-bit processor with an 8-bit external bus.

INTEL 80186 & 80188



- Introduced in 1982.
- They were 16-bit μ Ps.
- Clock speed was 6 MHz.
- 80188 was a cheaper version of 80186 with an 8-bit external data bus.

INTEL 80286



- Introduced in 1982.
- It was 16-bit μ P.
- Its clock speed was 8 MHz.
- Its data bus is 16-bit and address bus is 24-bit.
- It could address 16 MB of memory.
- It had 1,34,000 transistors.

32~BIT MICROPROCESSORS

INTEL 80386



- Introduced in 1986.
- It was first 32-bit μ P.
- Its data bus is 32-bit and address bus is 32-bit.
- It could address 4 GB of memory.
- It had 2,75,000 transistors.
- Its clock speed varied from 16 MHz to 33 MHz depending upon the various versions.

INTEL 80486



- Introduced in 1989.
- It was also 32-bit μ P.
- It had 1.2 million transistors.
- Its clock speed varied from 16 MHz to 100 MHz depending upon the various versions.
- 8 KB of cache memory was introduced.

INTEL PENTIUM



- Introduced in 1993.
- It was also 32-bit μ P.
- It was originally named 80586.
- Its clock speed was 66 MHz.
- Its data bus is 32-bit and address bus is 32-bit.

INTEL PENTIUM PRO



- Introduced in 1995.
- It was also 32-bit μ P.
- It had 21 million transistors.
- Cache memory:
 - 8 KB for instructions.
 - 8 KB for data.

INTEL PENTIUM II

- Introduced in 1997.
- It was also 32-bit μ P.
- Its clock speed was 233 MHz to 500 MHz.
- Could execute 333 million instructions per second.



INTEL PENTIUM II XEON



- Introduced in 1998.
- It was also 32-bit μ P.
- It was designed for servers.
- Its clock speed was 400 MHz to 450 MHz.

INTEL PENTIUM III



- Introduced in 1999.
- It was also 32-bit μ P.
- Its clock speed varied from 500 MHz to 1.4 GHz.
- It had 9.5 million transistors.

INTEL PENTIUM IV



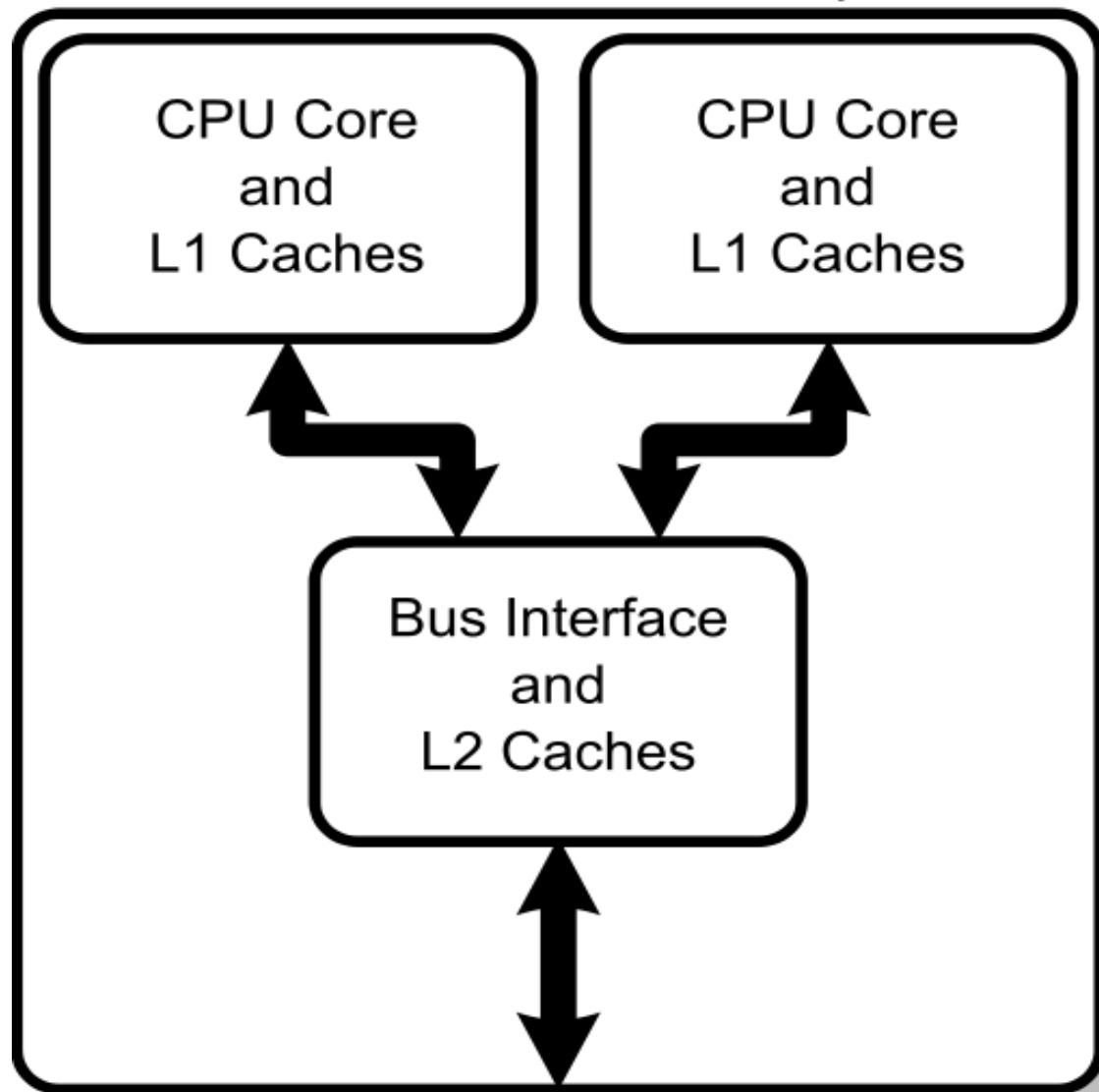
- Introduced in 2000.
- It was also 32-bit μ P.
- Its clock speed was from 1.3 GHz to 3.8 GHz.
- It had 42 million transistors.

INTEL DUAL CORE

- Introduced in 2006.
- It is 32-bit or 64-bit μ P.



Dual CPU Core Chip



64~BIT MICROPROCESSORS

Intel Core 2



Intel Core i3



INTEL CORE i5

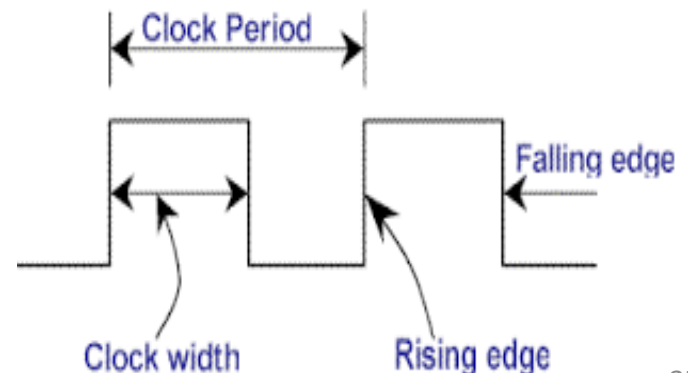


INTEL CORE i7



Basic Terms

- **Bit**: A digit of the binary number { 0 or 1 }
- **Nibble**: 4 bit **Byte**: 8 bit **word**: 16 bit
- **Double word**: 32 bit
- **Data**: binary number/code operated by an instruction
- **Address**: Identification number for memory locations
- **Clock**: square wave used to synchronize various devices in μP
- **Memory Capacity** = 2^n ,
 n->no. of address lines



BUS CONCEPT

- **BUS**: a collection of wires through which data is transmitted
 - Data: data, address or control signals
 - The size of bus (its width) is how many bits it can transfer at a time.

CLASSIFICATION OF BUSES:

1. **Address Bus**: sends a memory address along bus from the CPU to the memory. To fetch/write data, the CPU needs to tell the RAM the address
2. **ADDRESS BUS**: sends the actual data to and from the memory
3. **CONTROL BUS**: carries commands from the CPU and status messages from other hardware devices

BUS CONCEPT

- **BUS**: Group of conducting lines that carries data , address & control signals.

CLASSIFICATION OF BUSES:

1. **DATA BUS**: group of conducting lines that carries data.

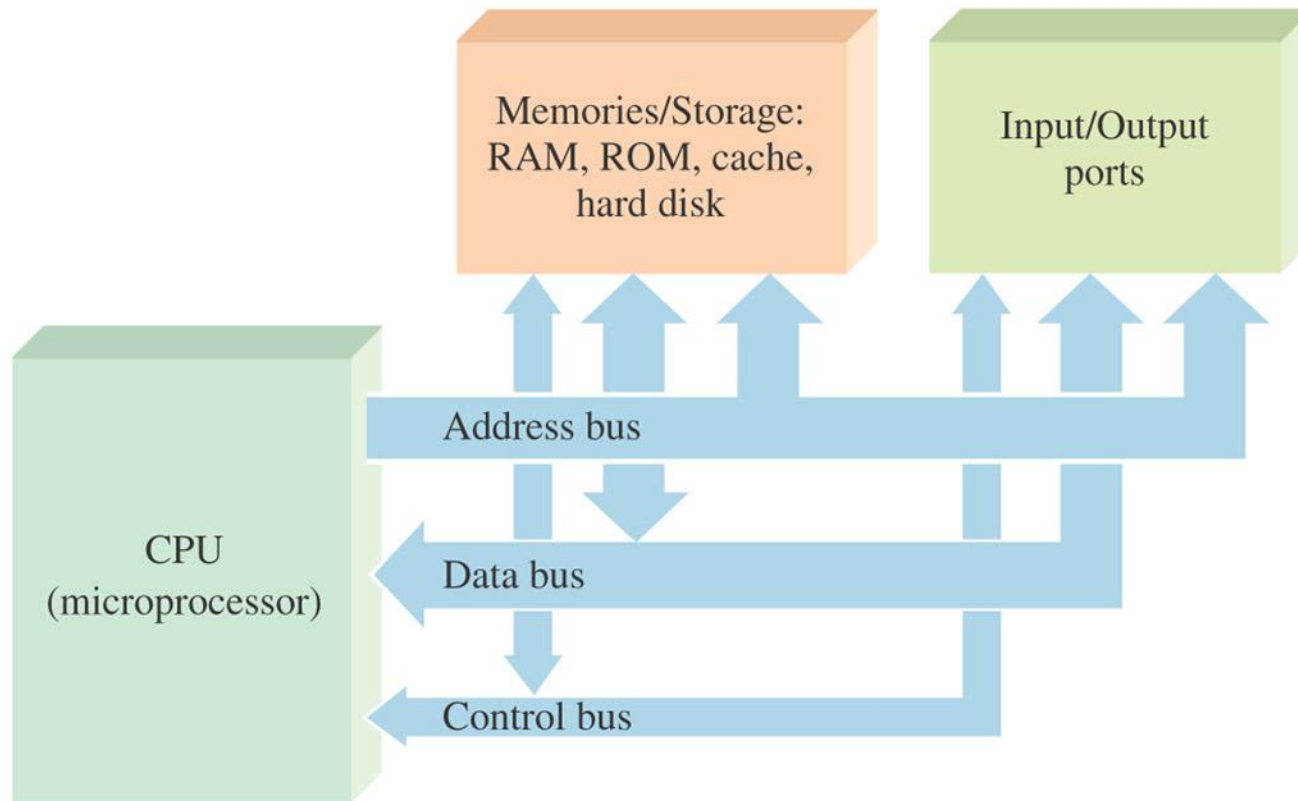
2. **ADDRESS BUS**: group of conducting lines that carries address.

3. **CONTROL BUS**: group of conducting lines that carries control signals {RD, WR etc}

CPU BUS: group of conducting lines that directly connected to μP

SYSTEM BUS: group of conducting lines that carries data , address & control signals in a μP system

Microcomputer CONCEPT



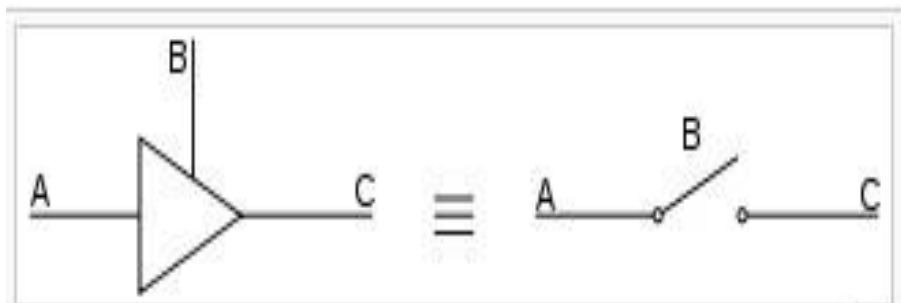
TRISTATE LOGIC

3 logic levels are:

- High State (logic 1)
- Low state (logic 0)
- High Impedance state

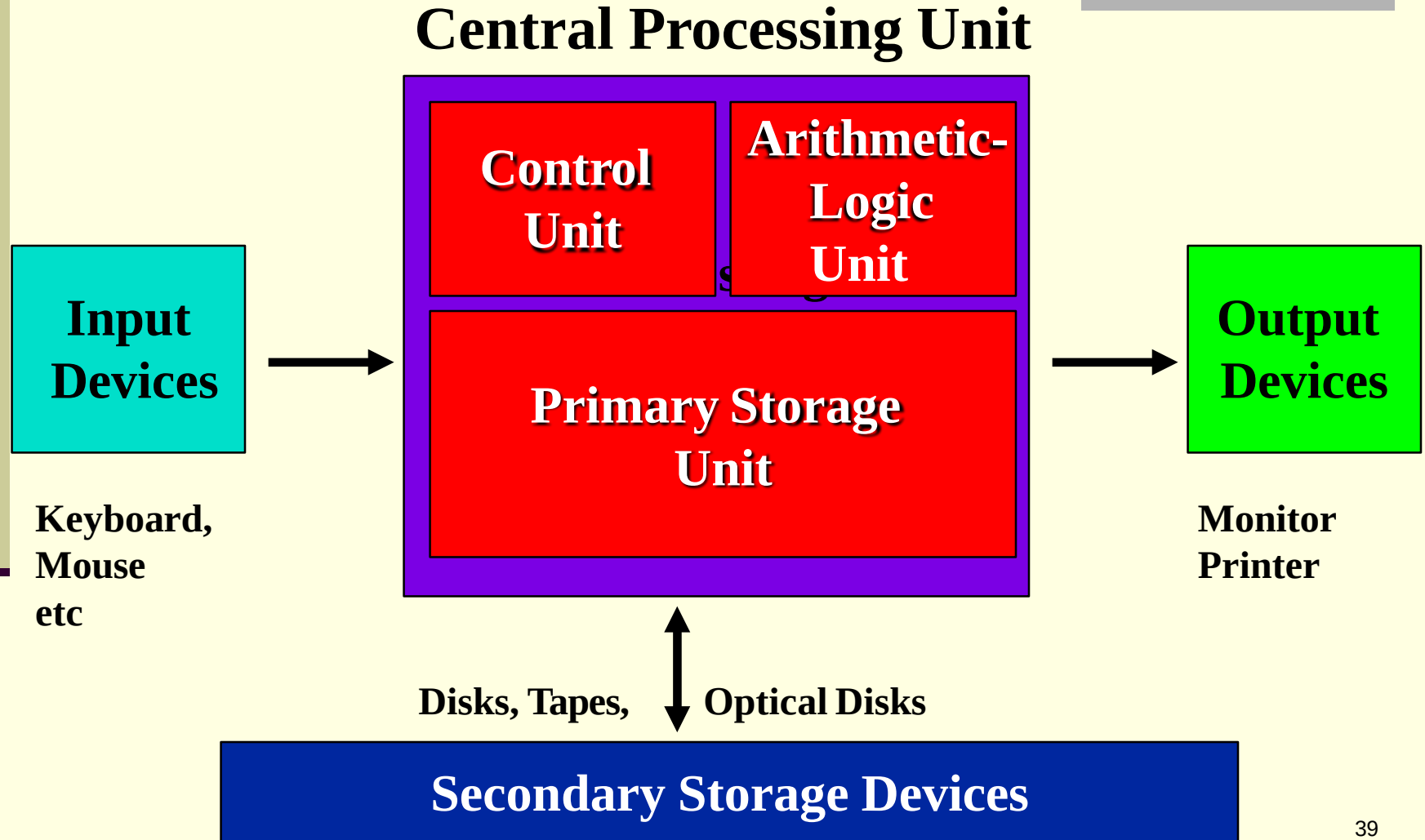
High Impedance: output is not being driven to any **defined** logic level by the output circuit.

INPUT		OUTPUT
A	B	C
0	1	0
1	1	1
X	0	Z(high impedance)



A tristate buffer can be thought of as a switch. If B is on, the switch is closed. If B is off, the switch is open.

Basic Microprocessors System



UNIT

2

THE 8086 MICROPROCESSOR

UNIT 2 Syllabus

- Introduction to 8086 – Features
- Memory and Memory Segmentations
- Microprocessor architecture
 - Bus Interface Unit
 - ❖ Instruction Queue
 - ❖ Segment Registers
 - ❖ Instruction Pointer
 - ❖ Address Adder (Address Summing Block
 - Execution Unit
 - ❖ Instruction Decoder
 - ❖ Control System
 - ❖ Arithmetic Logic Unit
 - ❖ General Purpose Registers
 - ❖ Flag Register
 - ❖ Pointer & Index registers

8086 Microprocessor-features

- INTEL launched 8086 in 1978
- 8086 is a 16-bit microprocessor.
- 8086 has a 20 bit address bus can access up to $2^{20} = 10,48,576$ bytes memory locations (1 MB).
- It provides 14, 16 -bit registers.
- Word size is 16 bits.
- It requires single phase clock with 33% duty cycle to provide internal timing.
- It can support upto **64K** I/O ports

8086 Microprocessor-features cont'd

- 8086 is designed to operate in **two modes**, Minimum and Maximum.
- It can prefetches up to **6** instruction bytes from memory and queues them in order to speed up instruction execution.
- It requires **+5V** power supply.
- A **40 pin** dual in line package.
- Address ranges from **00000H** to **FFFFFFH**
- Memory is byte addressable - Every byte has a separate address.

8086 Microprocessor-features cont'd

- **8086 is a 16-bit microprocessor with**
 - **16-bit Data Bus {D₀-D₁₅}**
 - **20-bit Address Bus {A₀-A₁₉} [can access up to $2^{20} = 1,048,576$ bytes = **1 MB** memory locations] .**
- **It has multiplexed address and data bus AD₀-AD₁₅ and A₁₆–A₁₉.**

8086 Microprocessor

- **8086 requires one phase clock with a 33% duty cycle to provide optimized internal timing.**
 - **Range of clock:**
 - **5 MHz for 8086**
 - **8Mhz for 8086-2**
 - **10Mhz for 8086-1**

Memory (RAM)

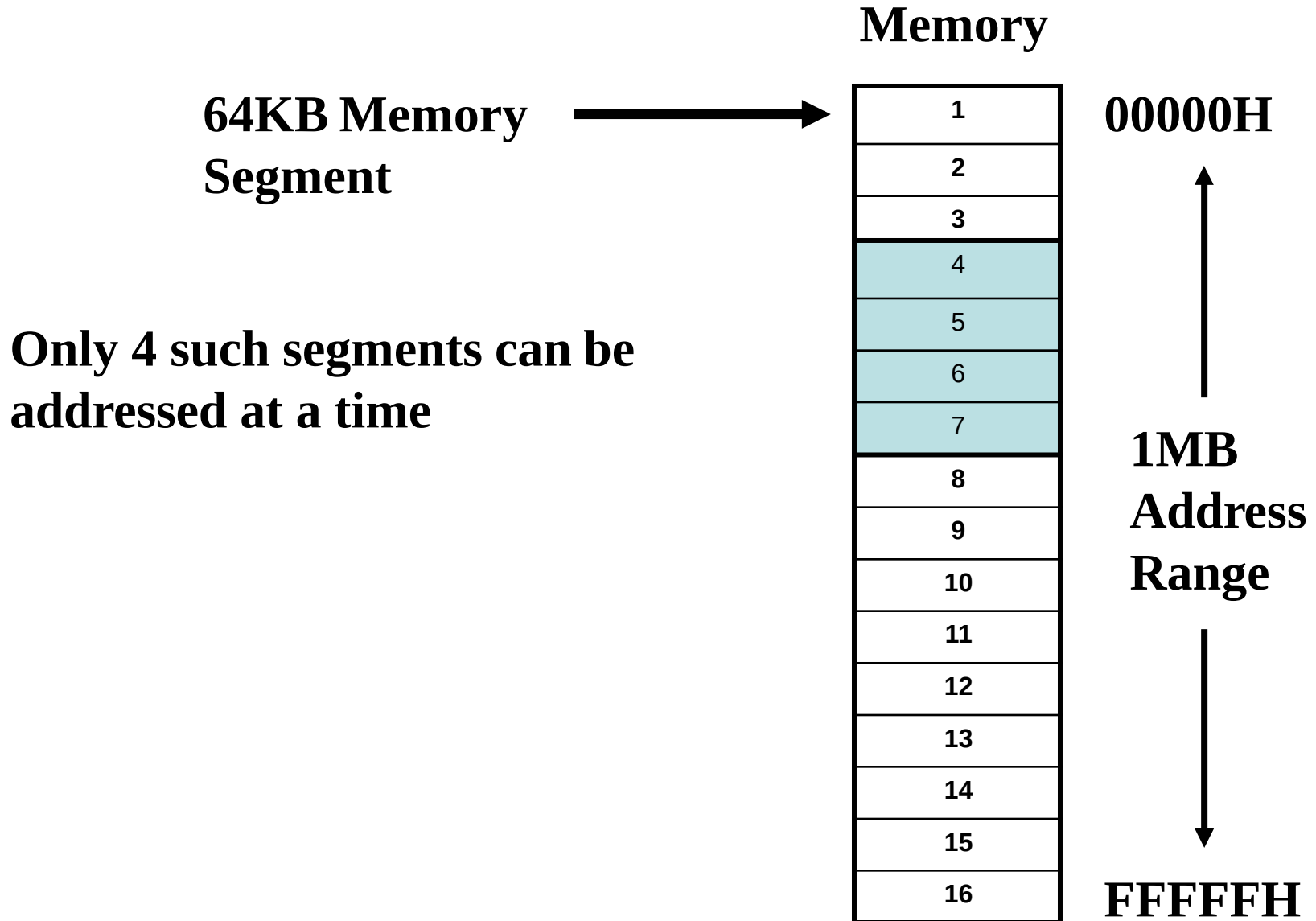
- In memory, data is stored as **bytes**.
- Each **byte** has a **specific address**.
- Intel 8086 has 20 lines address bus.
- With 20 address lines, the memory that can be addressed is 2^{20} bytes.
 - $2^{20} = 1,048,576$ bytes (1 MB).
- 8086 can access memory with address ranging from 00000 H to FFFFF H.

Memory Segmentation

- **Segmentation** is the process in which the **main memory** of the computer is **logically divided into different segments** and each segment has its own **base address**.
- The total memory size is divided into segments of various sizes.
- Segment is just an area in memory.
- The process of dividing memory this way is called **Segmentation**.

Memory Segmentation

- 8086 has a **20-bit** address bus
- So it can address a maximum of **1MB** of memory
- 8086 can work with only **four 64KB segments** at a time within this 1MB range
- These four memory segments are called
 - **Code** segment
 - **Stack** segment
 - **Data** segment
 - **Extra** segment



Code Segment

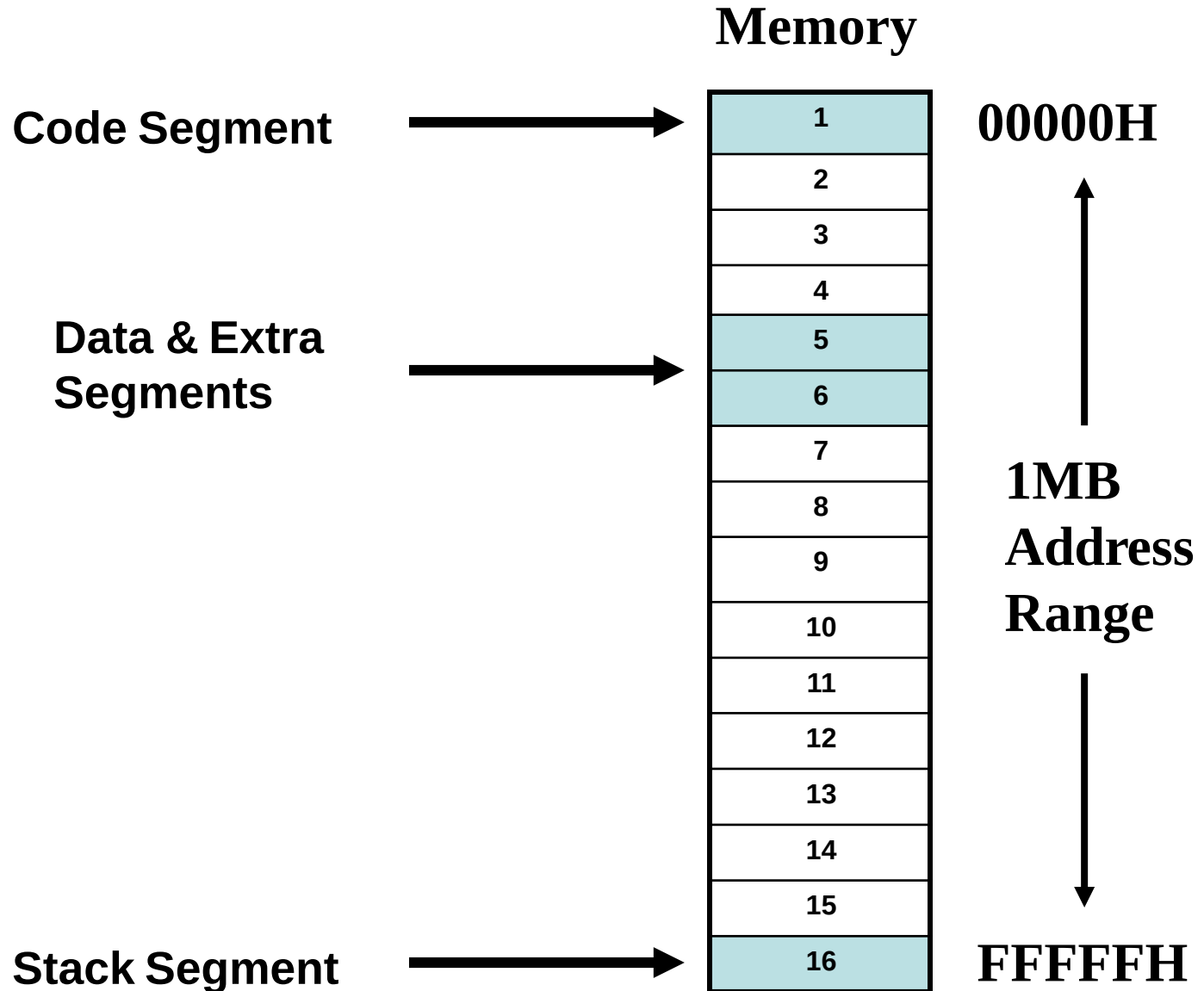
- That part of memory from where BIU is currently fetching instruction code bytes

Stack Segment

- A section of memory set aside to store addresses and data while a subprogram executes

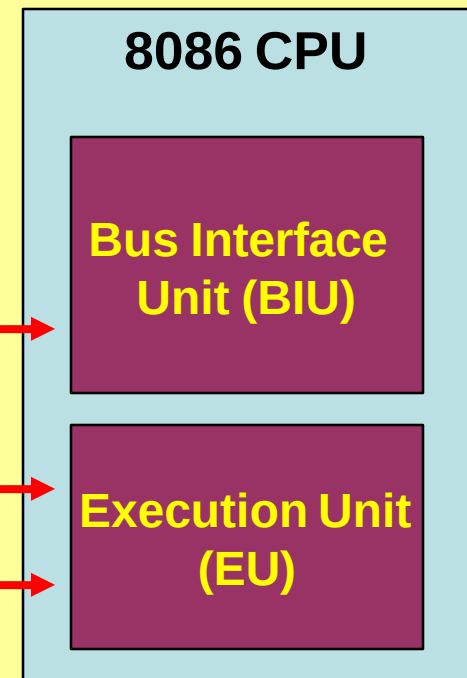
Data & Extra Segments

- Used for storing data values to be used in the program



8086 Internal Architecture

- 8086 employs parallel processing
- 8086 CPU has two parts which operate at the same time
 - Bus Interface Unit
 - Execution Unit
- CPU functions
 1. Fetch
 2. Decode
 3. Execute



8086 Internal Architecture

- 8086 has two blocks **BIU** and **EU**.
- The **BIU handles all transactions of data and addresses** on the buses for EU.
- The **BIU performs all bus operations** such as instruction fetching, reading and writing operands from/to memory and calculating the addresses of the memory operands. The instruction bytes are transferred to the **instruction queue**.
- **EU executes instructions** from the instruction system byte queue.

Bus Interface Unit

- Sends out addresses for memory locations
- Fetches Instructions from memory
- Reads/Writes data to memory
- Sends out addresses for I/O ports
- Reads/Writes data from/to Input/Output ports

Execution Unit

- Tells BIU (addresses) from where to fetch instructions or data, decodes & Executes instructions
- Dividing the work between BIU & EU speeds up processing

Architecture Diagram of 8086

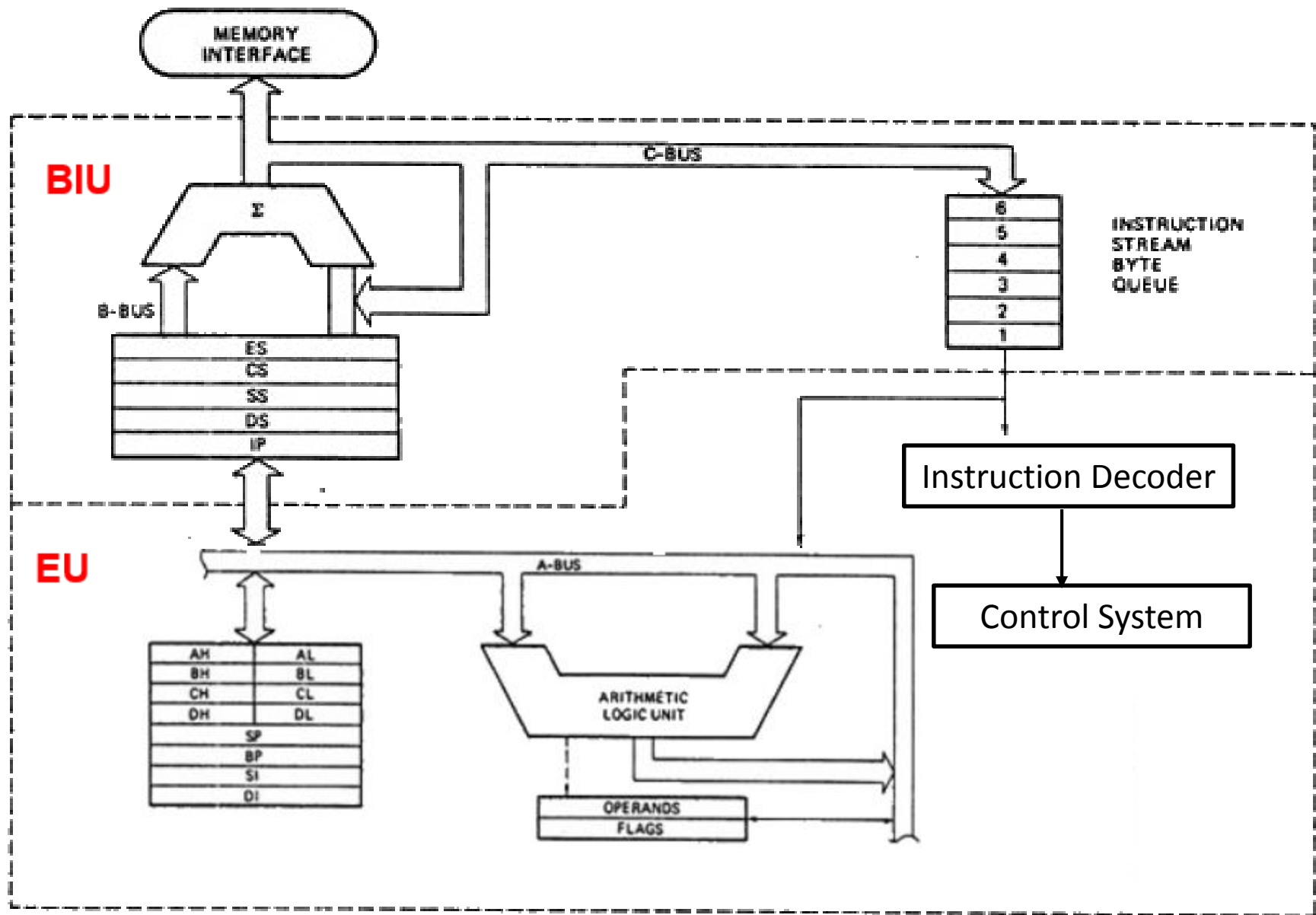
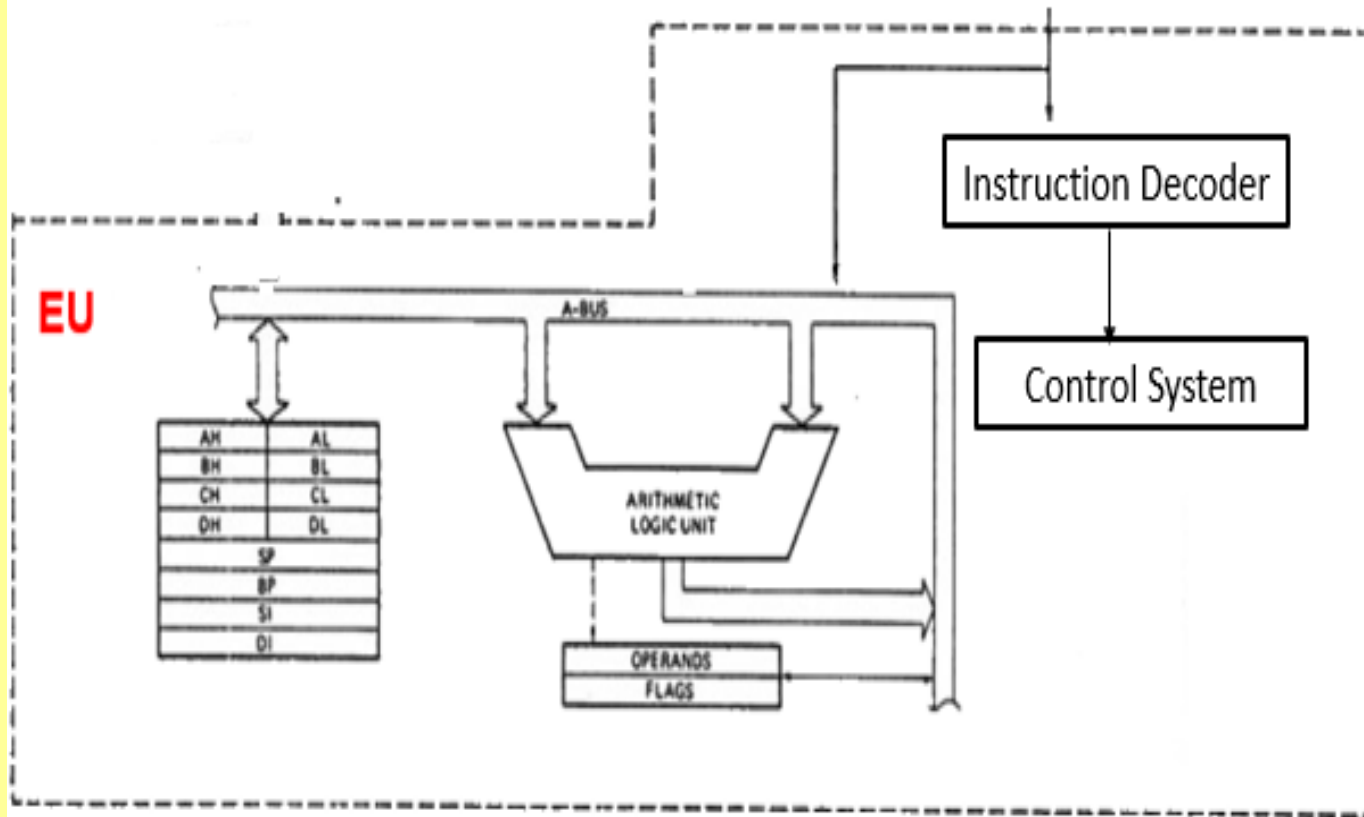


FIGURE 8086 internal block diagram. (Intel Corp.)

Execution Unit

- Main components are
 - **Instruction Decoder**
 - **Control System**
 - **Arithmetic Logic Unit**
 - **General Purpose Registers**
 - **Flag Register**
 - **Pointer & Index registers**

Execution Unit



Instruction Decoder

- **Translates instructions fetched from memory into a series of actions which EU carries out**

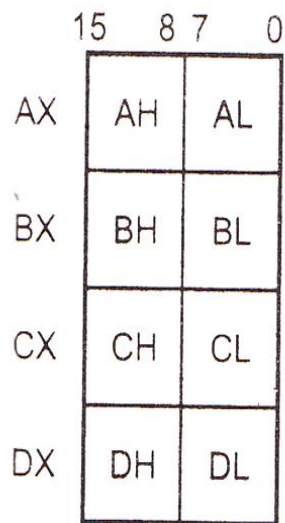
Control System

- **Generates timing and control signals to perform the internal operations of the microprocessor (Directs the internal operations)**

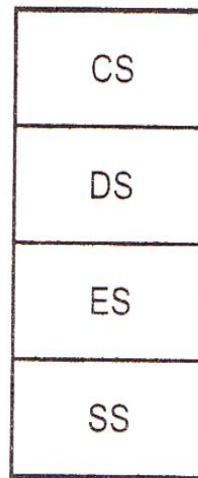
Arithmetic Logic Unit

- **EU has a 16-bit ALU which can ADD, SUBTRACT, AND, OR, increment, decrement, complement or shift binary numbers**

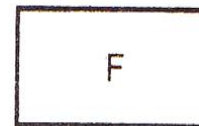
Registers



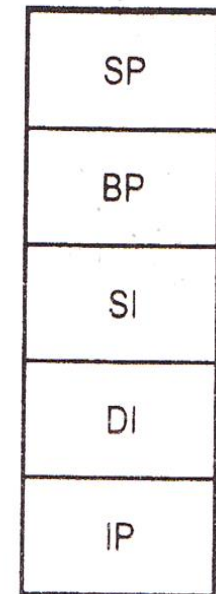
(a) General purpose registers



(b) Segment registers



(c) Flag registers



(d) Pointer and index registers

Registers

AX	AH	AL	Accumulator
BX	BH	BL	Base
CX	CH	CL	Count
DX	DH	DL	Data

} Data group

AX, BX, CX and DX are two bytes wide and each byte can be accessed separately

SP	Stack pointer
BP	Base pointer
SI	Source index
DI	Destination index
IP	Instruction pointer

} Pointer and index group

These registers are used as memory *pointers*.

Flags _H	Flags _L	Status and control flags
--------------------	--------------------	--------------------------

Flags will be discussed later

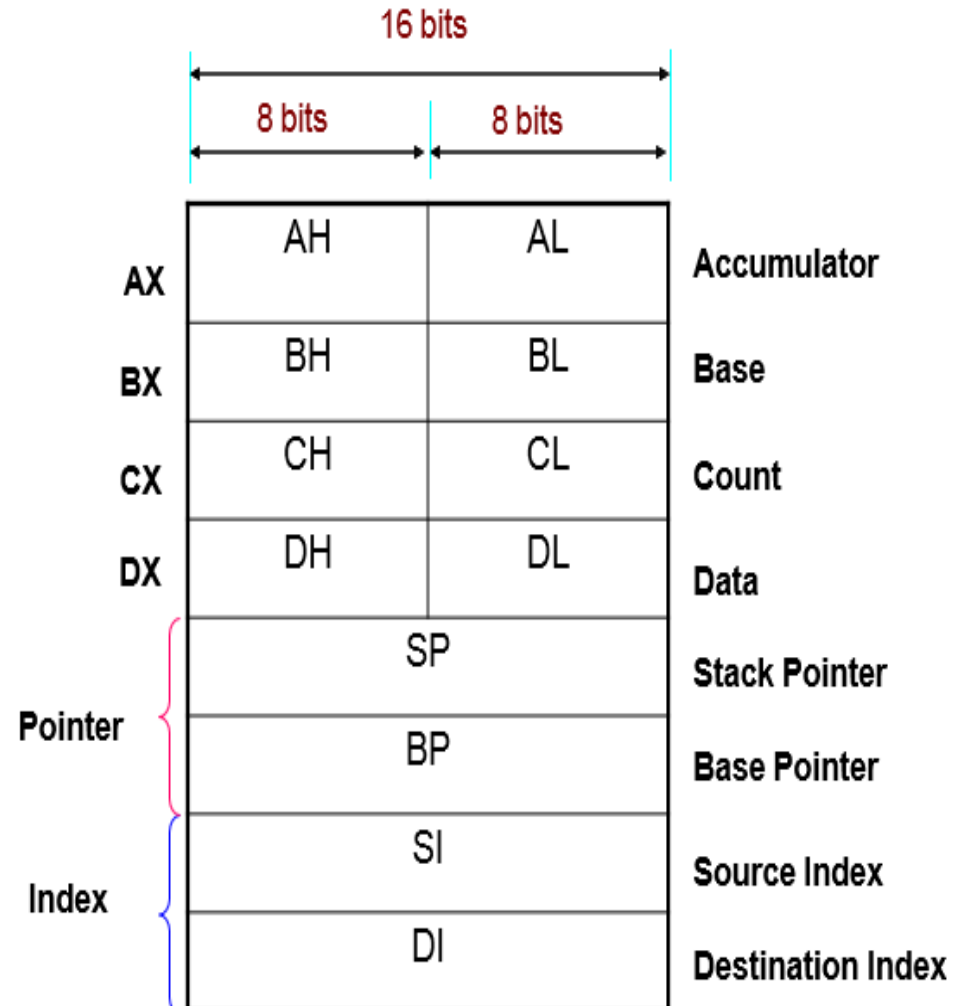
ES	Extra
CS	Code
DS	Data
SS	Stack

} Segment group

Segment registers are used as base address for a segment in the 1 M byte of memory

General Purpose Registers

- EU has 8 general purpose registers
- are used to store temporary data in the time of different operations in microprocessor.
- AL register is also called Accumulator
- Two registers can also be combined to form 16-bit registers
- The valid register pairs are – AX, BX, CX, DX



General Purpose Registers

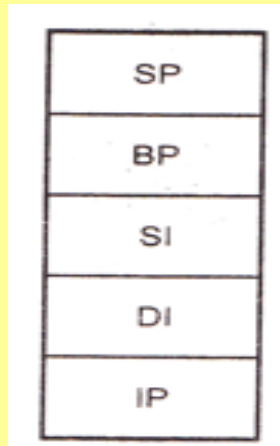
- **AX - the accumulator register (divided into AH / AL):**
 - Accumulator Register
 - Preferred register to use in arithmetic, logic and data transfer instructions because it generates the shortest Machine Language Code
 - Must be used in multiplication and division operations
 - Must also be used in I/O operations
- **BX - the base address register (divided into BH / BL). Also serves as an address Reg.**

General Purpose Registers

- **CX - the count register (divided into CH / CL): Counter Register**
 - Iterative code segments using the LOOP instruction
 - Repetitive operations on strings with the REP command
 - Count (in CL) of bits to shift and rotate
- **DX - the data register (divided into DH / DL):**
 - DX:AX concatenated into 32-bit register for some MUL and DIV operations
 - Also used in I/O operations.

Pointers and Index Registers

- Used to **Keep offset addresses.**
- Used in **various forms of memory addressing.**



- The index registers (SI & DI) and the BX generally default to the Data segment register (DS).

Pointers and Index Registers

➤ SI - source index register:

- Can be used for pointer addressing of data
- Used as source in some string processing instructions
- Offset address relative to DS

➤ DI - destination index register:

- Can be used for pointer addressing of data
- Used as destination in some string processing instructions
- Offset address relative to ES

Pointers and Index Registers

➤ **SI - source index register:**

- is required for some string operations
- When string operations are performed, the SI register points to memory locations in the data segment which is addressed by the DS register. Thus, SI is associated with the DS in string operations.

➤ **DI - destination index register:**

- is also required for some string operations.
- When string operations are performed, the DI register points to memory locations in the data segment which is addressed by the ES register. Thus, DI is associated with the ES in string operations.
- The SI and the DI registers may also be used to access data stored in arrays

Pointers and Index Registers

➤ **BP - base pointer:**

- Primarily used to access parameters (data) passed via the stack
- Offset address relative to SS
- Can be used to access data in other segments

➤ **SP - stack pointer:**

- Always points to top item on the stack
- Offset address relative to SS
- Always points to word (byte at even address)
- An empty stack will had $SP = FFFEh$

Flag Register

- A flag register a flip-flop which indicates some condition produced by the execution of instruction or controls certain operations of the EU (Execution Unit)
- 8086 has a **16-bit** flag register
- Contains **9** active flags

Flag Register

- 8086 has a **16-bit** flag register
- Contains **9** active flags
- There are two types of flags in 8086
 - **Conditional** flags – **six** flags, set or reset by EU on the basis of results of some arithmetic operations
 - **Control** flags – **three** flags, used to control certain operations of the processor

Flag Register

U	U	U	U	OF	DF	IF	TF	SF	ZF	U	AF	U	PF	U	CF
---	---	---	---	----	----	----	----	----	----	---	----	---	----	---	----

1.	CF	CARRY FLAG	Conditional Flags (Compatible with 8085, except OF)
2.	PF	PARITY FLAG	
3.	AF	AUXILIARY CARRY	
4.	ZF	ZERO FLAG	
5.	SF	SIGN FLAG	
6.	OF	OVERFLOW FLAG	
7.	TF	TRAP FLAG	Control Flags
8.	IF	INTERRUPT FLAG	
9.	DF	DIRECTION FLAG	

Flag Register

Auxiliary Carry Flag

This is set, if there is a carry from the lowest nibble, i.e, bit three during addition, or borrow for the lowest nibble, i.e, bit three, during subtraction.

Carry Flag

This flag is set, when there is a carry out of MSB in case of addition or a borrow in case of subtraction.

Sign Flag

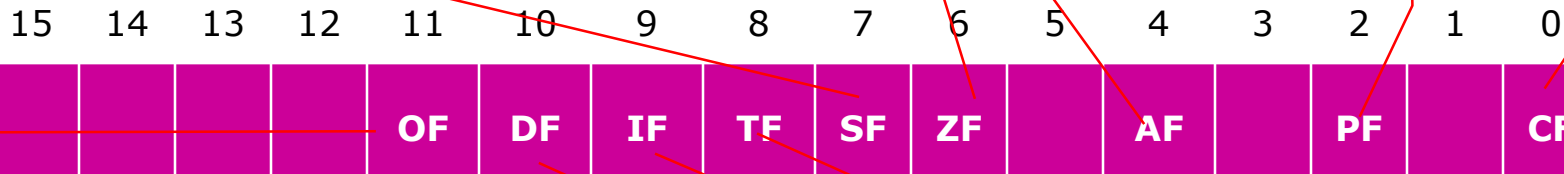
This flag is set, when the result of any computation is negative

Zero Flag

This flag is set, if the result of the computation or comparison performed by an instruction is zero

Parity Flag

This flag is set to 1, if the lower byte of the result contains even number of 1's ; for odd number of 1's set to zero.



Over flow Flag

This flag is set, if an overflow occurs, i.e, if the result of a signed operation is large enough to accommodate in a destination register. The result is of more than 7-bits in size in case of 8-bit signed operation and more than 15-bits in size in case of 16-bit sign operations, then the overflow will be set.

Tarp Flag

If this flag is set, it will work in a single step mode. After each instruction, one internal interrupt is generated. It helps to execute some program instruction by instruction.

Direction Flag

This is used by string manipulation instructions. If this flag bit is '0', the string is processed beginning from the lowest address to the highest address, i.e., auto incrementing mode. Otherwise, the string is processed from the highest address towards the lowest address, i.e., auto decrementing mode.

Interrupt Flag

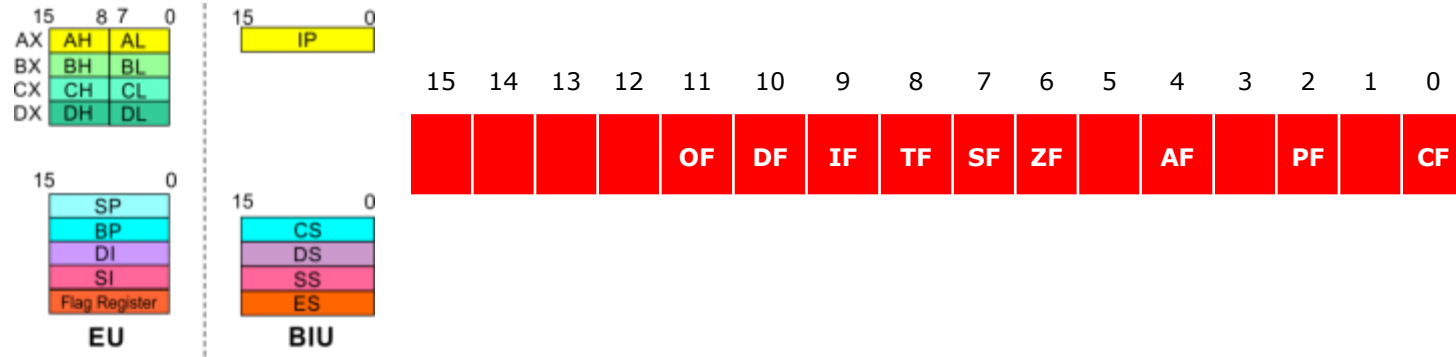
If IF = 1, then MPU will recognize the interrupts from peripherals. For IF = 0, the interrupts will be ignored

Flag Register

- **Six** of the flags are **status indicators** reflecting properties of the last arithmetic or logical instruction.
- For example, if register **AL = 7Fh** and the instruction **ADD AL,1** is executed then the following happen
 - **AL = 80h**
 - **CF = 0**; there is no carry out of bit 7
 - **PF = 0**; 80h has an odd number of ones
 - **AF = 1**; there is a carry out of bit 3 into bit 4
 - **ZF = 0**; the result is not zero
 - **SF = 1**; bit seven is one
 - **OF = 1**; the sign bit has changed

Registers, Flag

8086 registers categorized into 4 groups

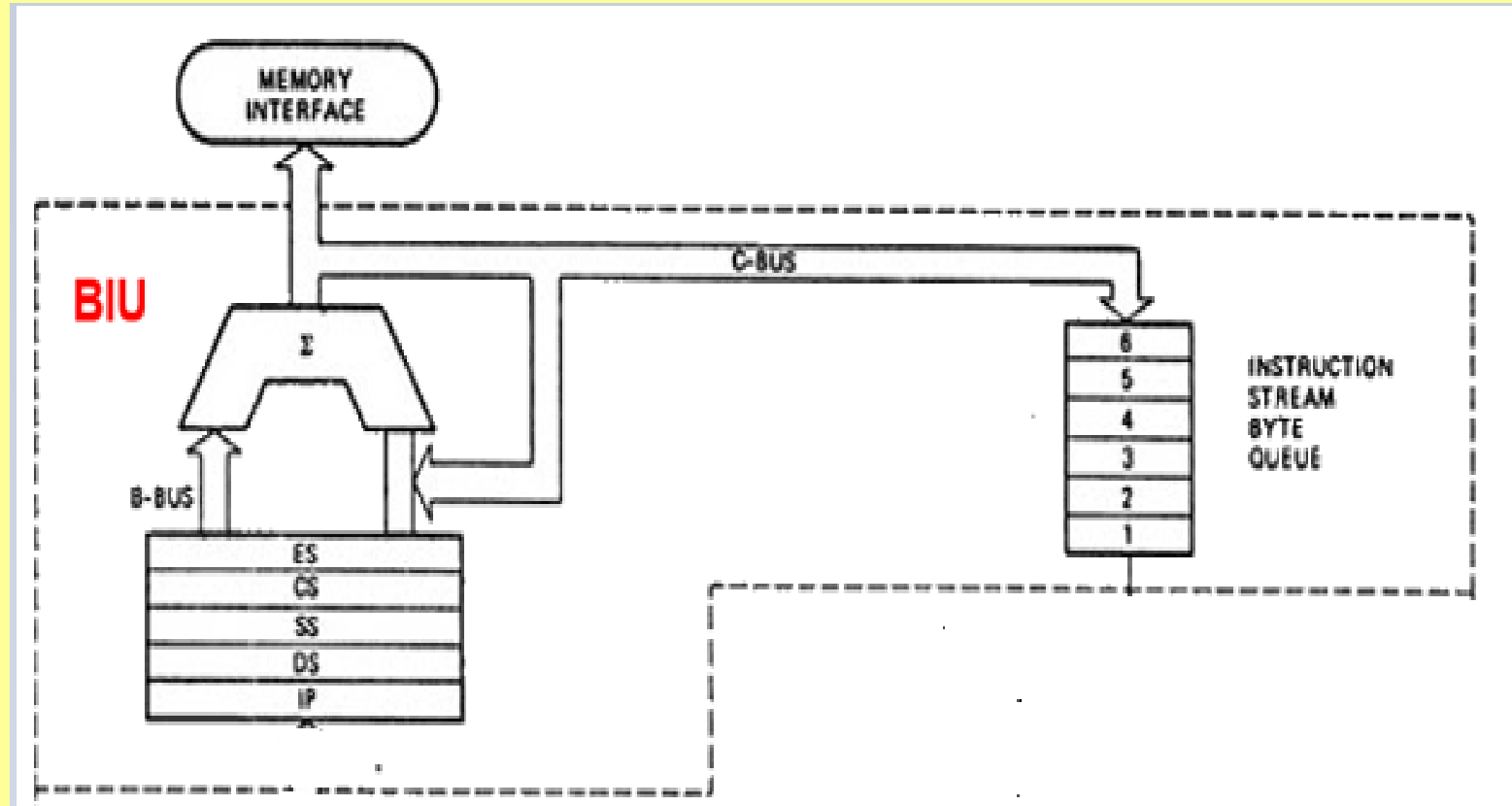


Sl.No.	Type	Register width	Name of register
1	General purpose register	16 bit	AX, BX, CX, DX
		8 bit	AL, AH, BL, BH, CL, CH, DL, DH
2	Pointer register	16 bit	SP, BP
3	Index register	16 bit	SI, DI
4	Instruction Pointer	16 bit	IP
5	Segment register	16 bit	CS, DS, SS, ES
6	Flag (PSW)	16 bit	Flag register

Registers and Special Functions

Register	Name of the Register	Special Function
AX	16-bit Accumulator	Stores the 16-bit results of arithmetic and logic operations
AL	8-bit Accumulator	Stores the 8-bit results of arithmetic and logic operations
BX	Base register	Used to hold base value in base addressing mode to access memory data
CX	Count Register	Used to hold the count value in SHIFT, ROTATE and LOOP instructions
DX	Data Register	Used to hold data for multiplication and division operations
SP	Stack Pointer	Used to hold the offset address of top stack memory
BP	Base Pointer	Used to hold the base value in base addressing using SS register to access data from stack memory
SI	Source Index	Used to hold index value of source operand (data) for string instructions
DI	Data Index	Used to hold the index value of destination operand (data) for string operations

Bus Interface Unit



Bus Interface Unit

- Main Components are
 - **Instruction Queue**
 - **Segment Registers**
 - **Instruction Pointer**
 - **Address Adder (Address Summing Block**

Instruction Queue

- 8086 employs parallel processing
- When EU is busy decoding or executing current instruction, the buses of 8086 **may not be** in use.
- At that time, BIU can use buses to fetch upto six instruction bytes for the following instructions
- BIU stores these pre-fetched bytes in a **FIFO** register called Instruction **Queue**
 - So that the execution unit gets the instructions for execution in the order they are fetched.
- When EU is ready for its next instruction, it simply reads the instruction from the queue in BIU

Pipelining

- EU of 8086 does not have to wait in between for BIU to fetch next instruction byte from memory
- So the presence of a queue in 8086 speeds up the processing
- Feature of fetching the next instruction while the current instruction executes is called **pipelining**

Memory Segmentation

- **Segmentation** is the process in which the **main memory** of the computer is **logically divided into different segments** and each segment has its own **base address**. It is basically used to enhance the speed of execution of the computer system, so that the processor is able to fetch and execute the data from the memory easily and fast.

Memory Segmentation

- 8086 has a **20-bit** address bus
- So it can address a maximum of **1MB** of memory
- 8086 can work with only **four 64KB segments** at a time within this 1MB range
- These four memory segments are called
 - **Code** segment
 - **Stack** segment
 - **Data** segment
 - **Extra** segment

Memory

**64KB Memory
Segment**



1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16

00000H



**1MB
Address
Range**



FFFFFFH

**Only 4 such segments can be
addressed at a time**

Code Segment

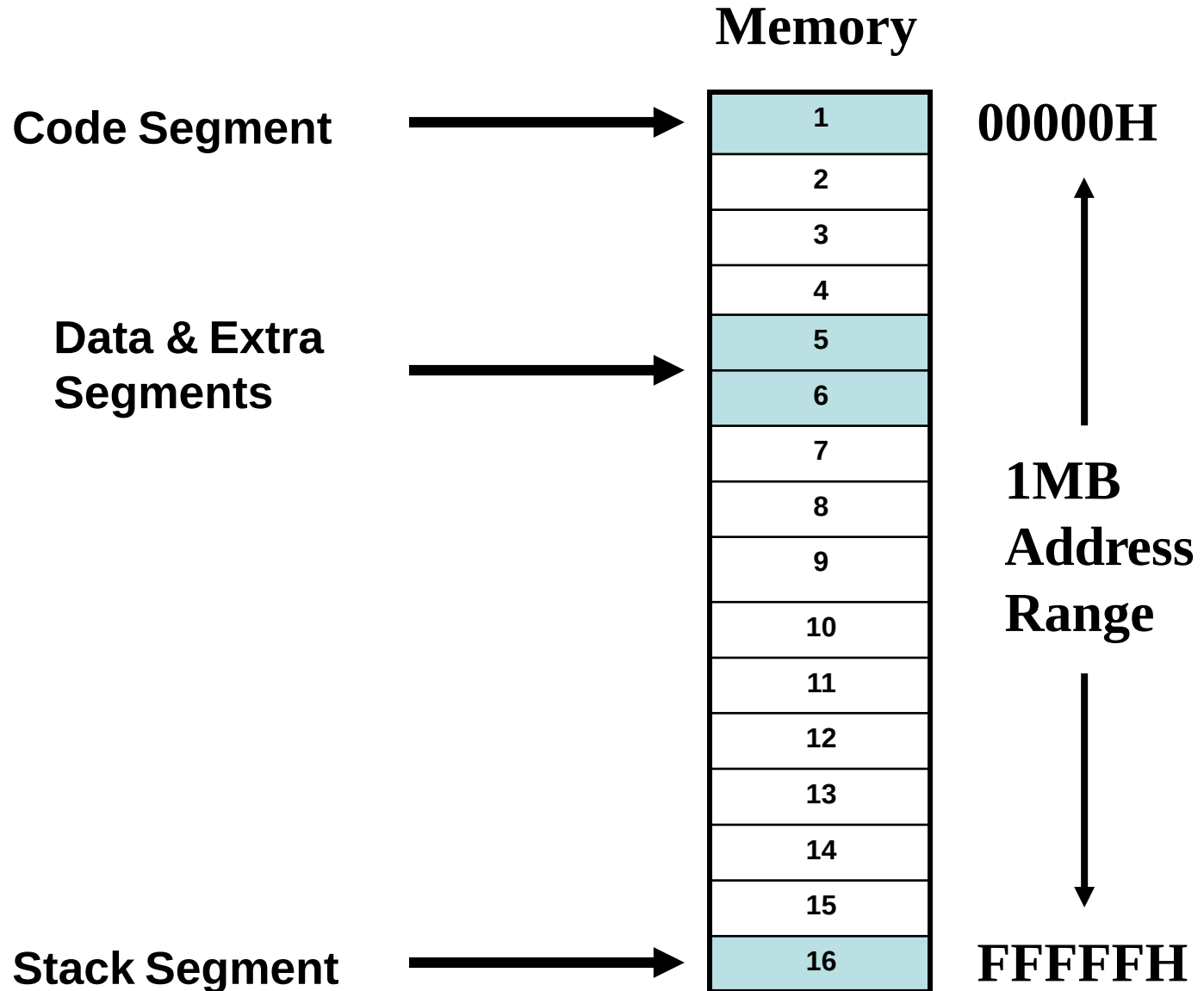
- That part of memory from where BIU is currently fetching instruction code bytes

Stack Segment

- A section of memory set aside to store addresses and data while a subprogram executes

Data & Extra Segments

- Used for storing data values to be used in the program



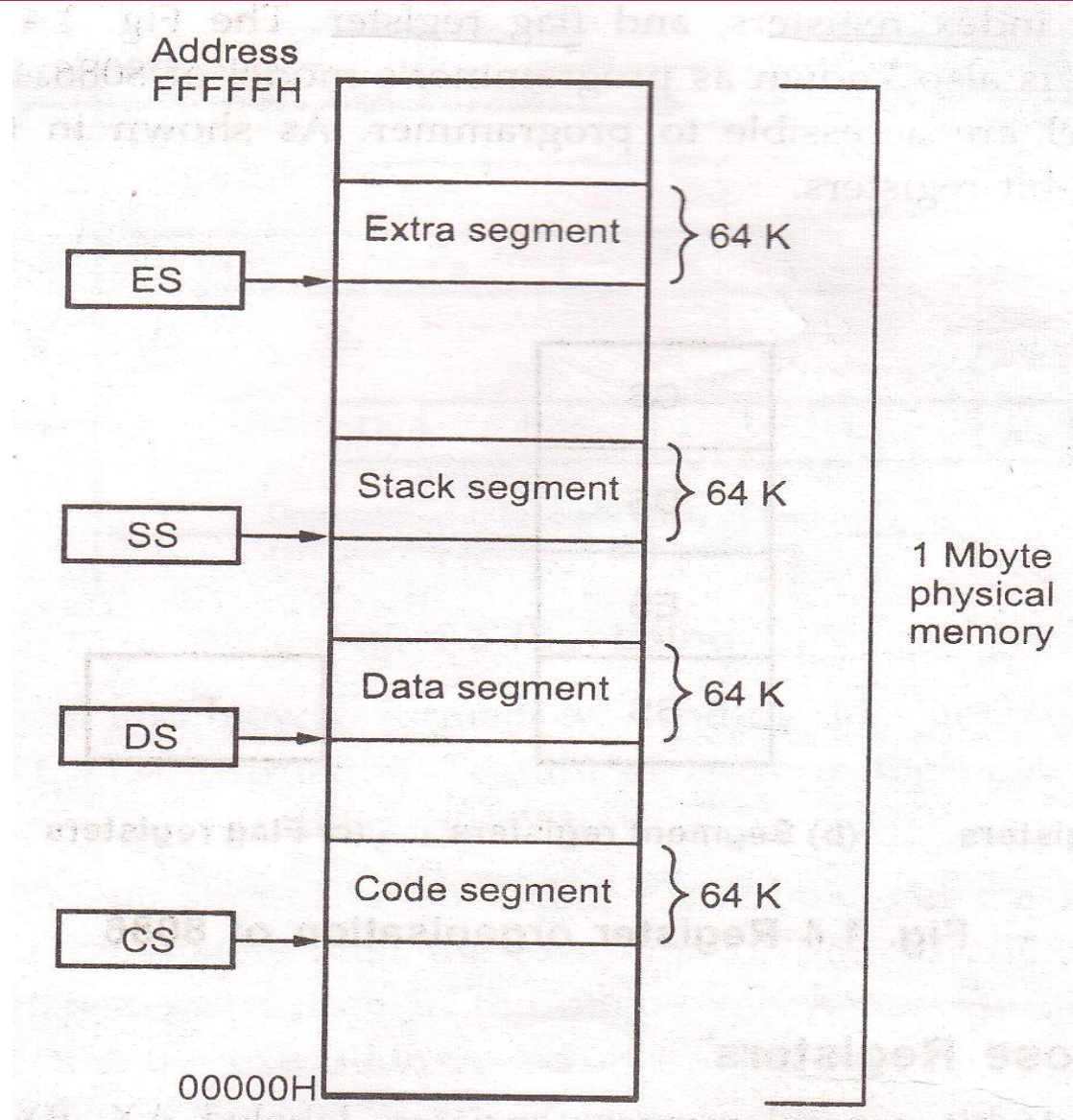
Segment Registers

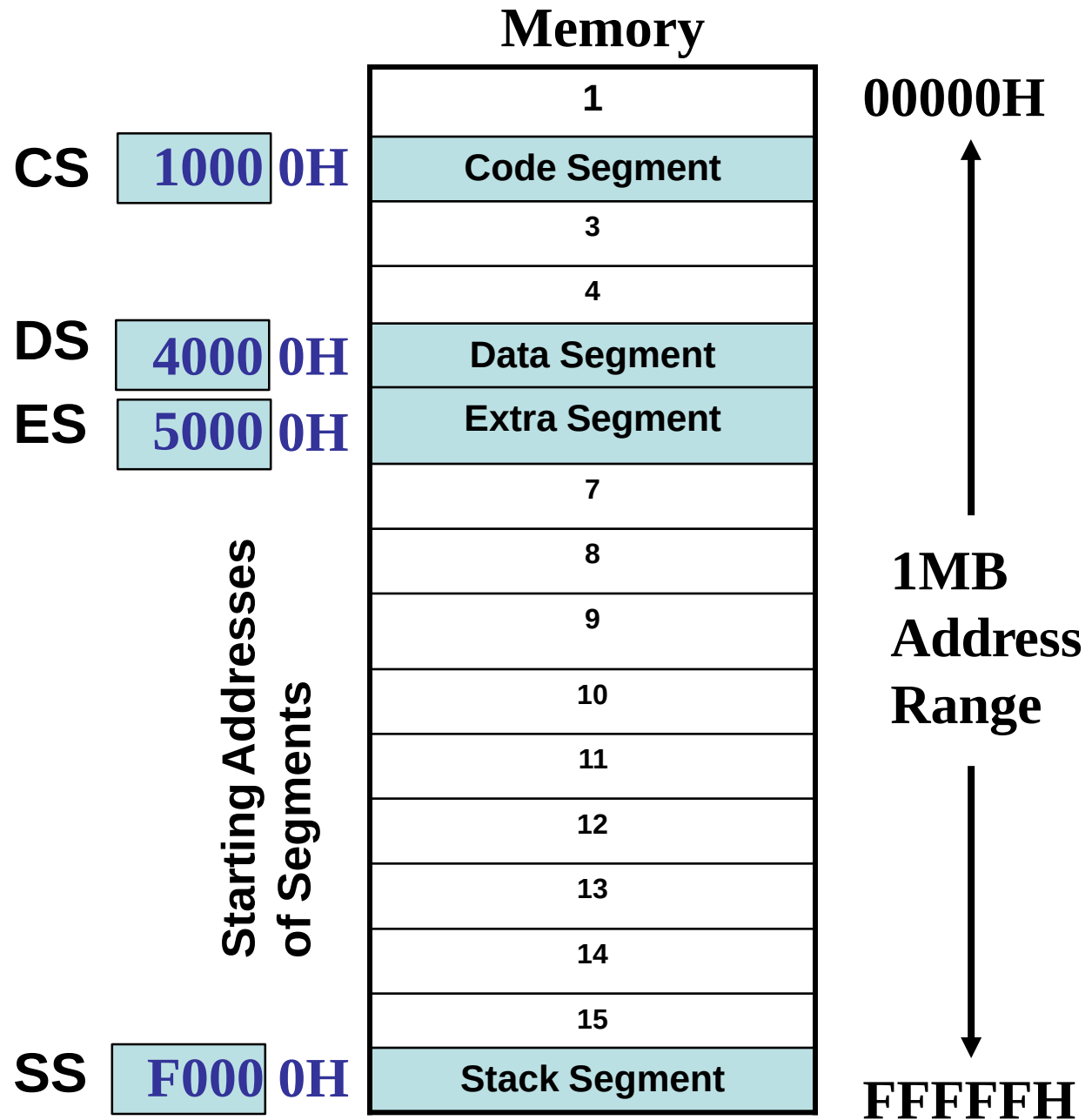
- Physical address of 8086 is 20 Bit wide (To access 1 Mb memory locations)
- **BUT**- But the registers and memory locations which contains logical address are 16 Bit
- Hence segmentation is required

Segment Registers

- hold the upper 16-bits of the starting address for each of the segments
- The four segment registers are
 - **CS (Code Segment register)**
 - **DS (Data Segment register)**
 - **SS (Stack Segment register)**
 - **ES (Extra Segment register)**

Segment Registers and Segmentation

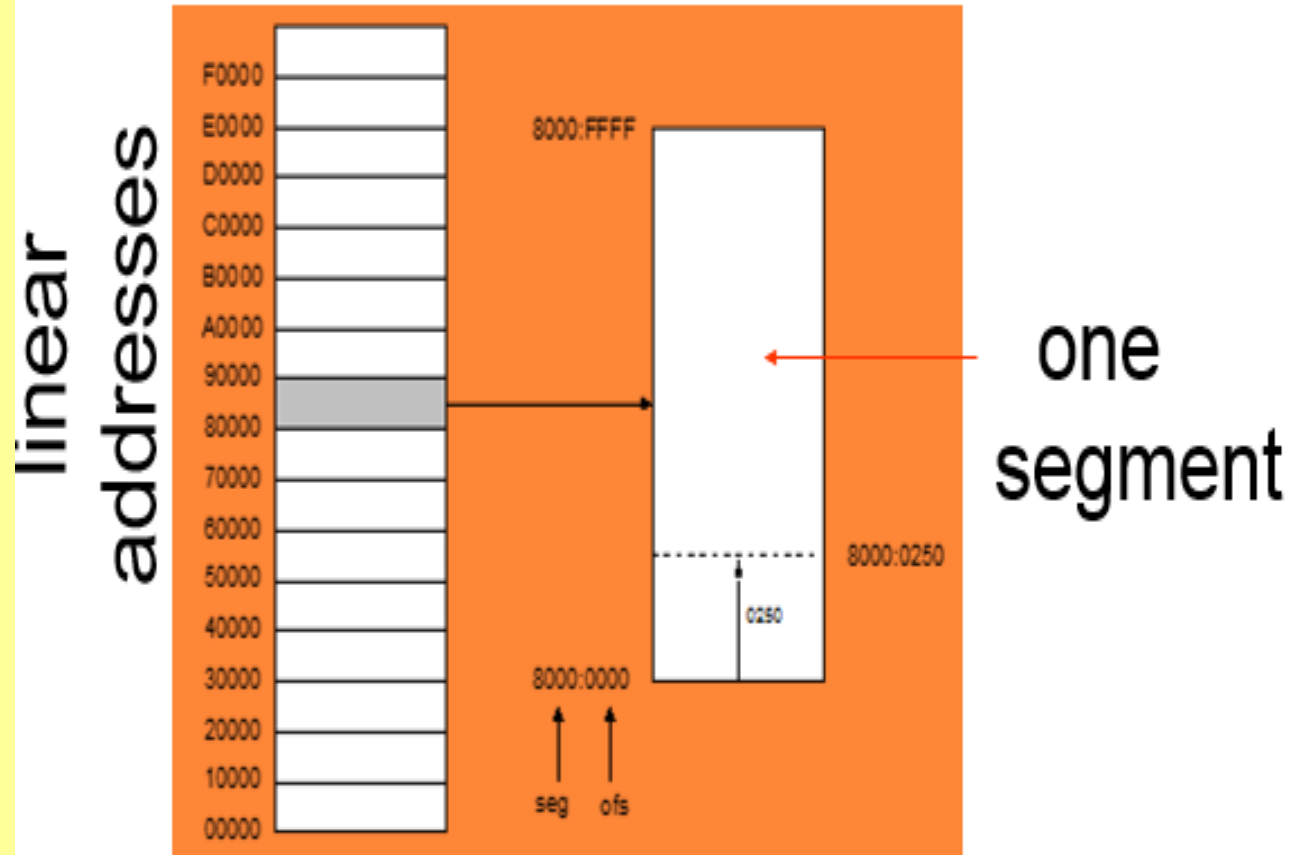


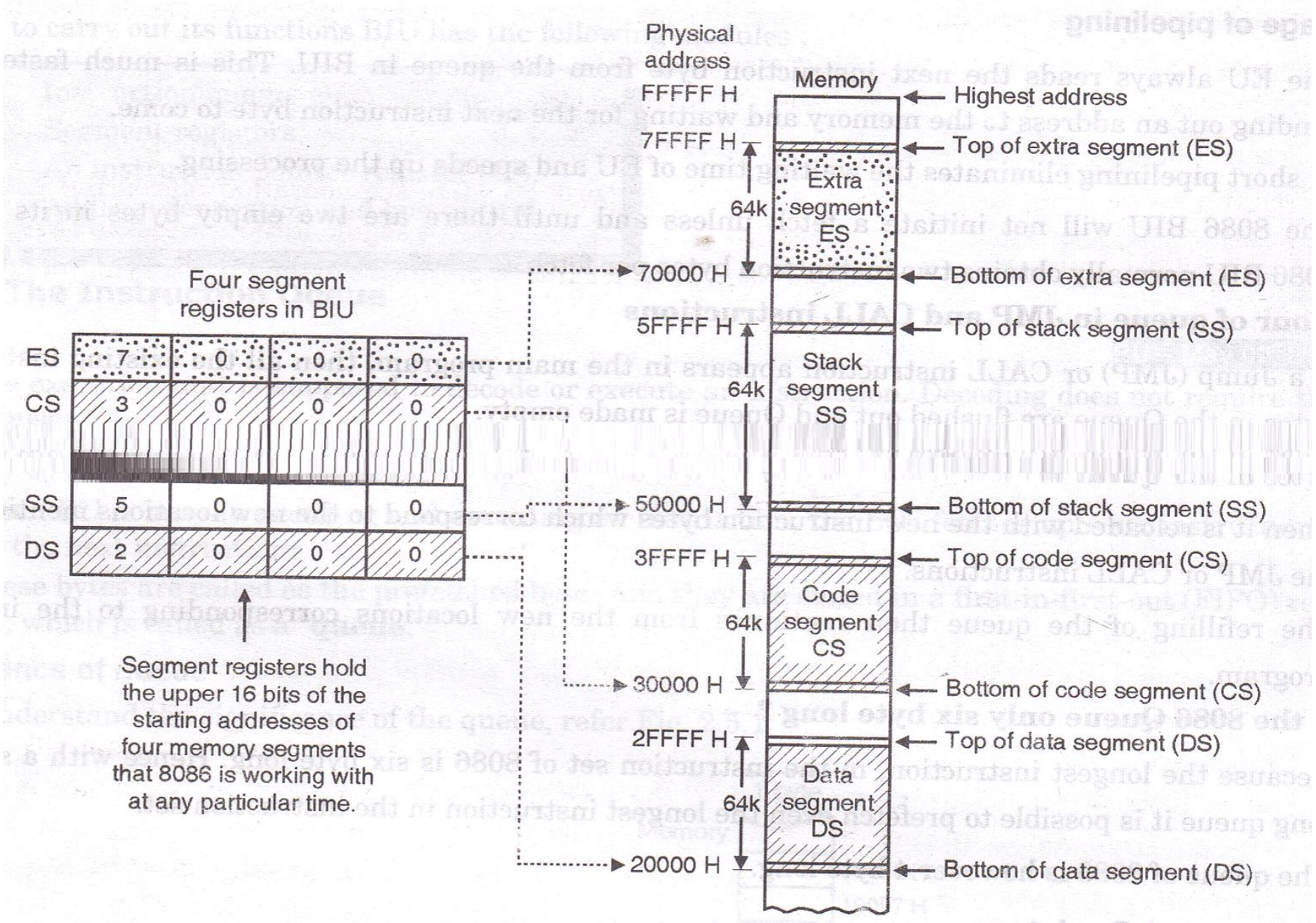


Max. size of Segment

- All offsets are limited to 16-bits.
- It means that the maximum size possible for segment is $2^{16}-1 = 65,535$ bytes (64 KB).
- The offset of the first location within the segment is 0000 H.
- The offset of the last location in the segment is FFFF H.

Max. size of Segment





- Address of a segment is of 20-bits
- A segment register stores only upper 16-bits
- BIU always inserts zeros for the lowest 4-bits of the 20-bit starting address.
- E.g. if CS = 348AH, then the code segment will start at 348A0H
- A 64-KB segment can be located anywhere in the memory, but will start at an address with zeros in the lowest 4-bits

Physical Address Generation

- In order to calculate the physical address, we need to master the concept of:
 - Memory Segmentation and the Segments
 - Segment Registers
 - Index and pointer Registers

Instruction Pointer and summing block

➤ Instruction Pointer (IP) Register

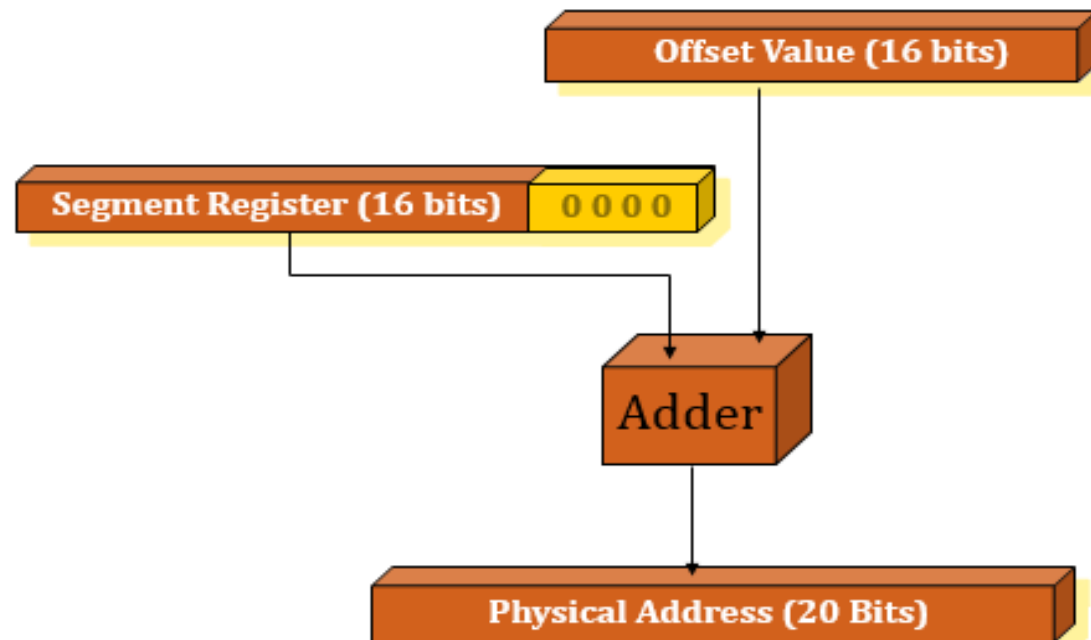
- a 16-bit register
- Holds 16-bit **offset**, of instruction in the **code segment** that is to be executed next.
- The IP always references the **Code segment register (CS)**.
- BIU uses **IP** and **CS** registers to generate the **20-bit address** of the instruction to be fetched from memory.

Instruction Pointer and summing block

- The value contained in the instruction pointer is called as an **offset** because this value must be added to the **base address** of the **code segment**, which is available in the CS register to find the **20-bit physical address**.
- The **value** of the instruction pointer is incremented after **executing every instruction**.
- To form a **20bit address** of the next instruction, the **16 bit address of the IP** is added (by the address summing block) to the **address contained in the CS**, which has been shifted four bits to the left.

Physical Address Generation

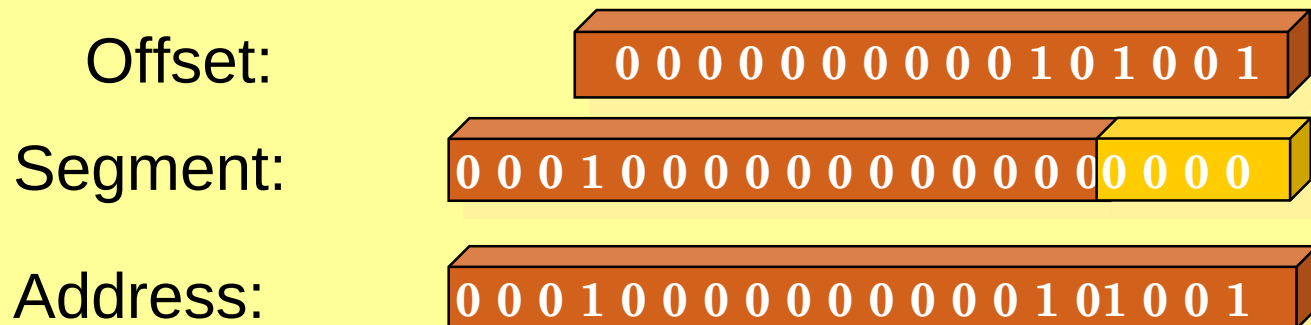
- The BIU has a dedicated adder for determining physical memory addresses



Physical Address Generation

➤ Example:

- If the data segment starts at location 1000h and a data reference contains the address 29h where is the actual data?



Physical Address Generation

- Logical Address is specified as **segment: offset**
- Physical address is obtained by **shifting the segment address 4 bits to the left and adding the offset address**
- Thus the physical address of the logical address A4FB:4872 is

A4FB0

+ 4872

A9822

Physical Address Generation

Example

- If DS=7FA2H and the offset is 438EH

a) Calculate the physical address

$$7FA20 + 438E = 83DAE$$

b) calculate the lower range

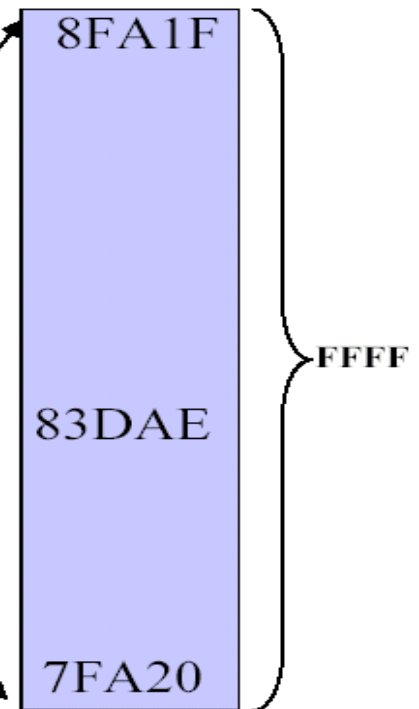
$$7FA20 + 0000 = 7FA20$$

c) Calculate the upper range of the data segment

$$7FA20 + FFFF = 8FA1F$$

d) Show the logical Address

7FA2:438E



Physical Address Calculation

Start of Code Segment

348A0H

IP = 4214H

Code Byte

38AB4H

MOV AL, BL

Memory

1

Data Segment

3

4

Code Segment

Extra Segment

7

8

9

10

11

12

13

14

15

Stack Segment

00000H

1MB
Address
Range

FFFFFH

CS 348A0H

IP + 4214H

Physical Address 38AB4H

Stack Segment (SS) Register

Stack Pointer (SP) Register

- Upper 16-bits of the starting address of stack segment is stored in SS register
- It is located in BIU
- SP register holds a 16-bit offset from the start of stack segment to the top of the stack
- It is located in EU

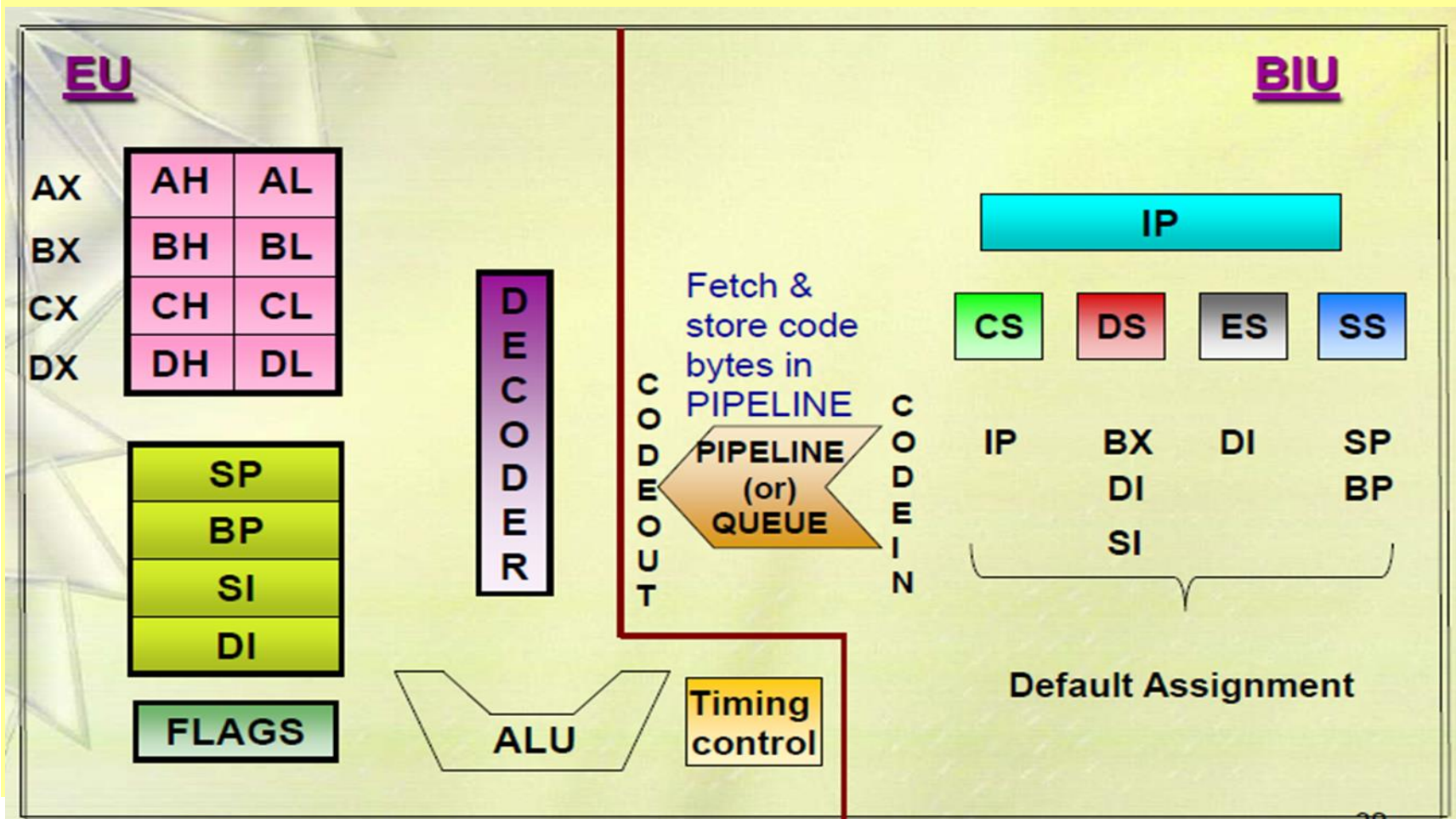
Other Pointer & Index Registers

- Base Pointer (BP) register
- Source Index (SI) register
- Destination Index (DI) register
- Can be used for temporary storage of data
- Main use is to hold a 16-bit offset of a data word in one of the segments

Segment and Address register combination

- **CS:IP**
- **SS:SP SS:BP**
- **DS:BX DS:SI**
- **DS:DI (for other than string operations)**
- **ES:DI (for string operations)**

Summary of Registers & Pipeline of 8086 μ P



Where to Look for the Offset

Segment	Offset Registers	Function
CS	IP	Address of the next instruction
DS	BX, DI, SI	Address of data
SS	SP, BP	Address in the stack
ES	BX, DI, SI	Address of destination data (for string operations)

Question

- ⦿ The contents of the following registers are:
 - ⦿ CS = 1111 H
 - ⦿ DS = 3333 H
 - ⦿ SS = 2526 H
 - ⦿ IP = 1232 H
 - ⦿ SP = 1100 H
 - ⦿ DI = 0020 H
- ⦿ Calculate the corresponding physical addresses for the address bytes in CS, DS and SS.

Solution

1. CS = 1111 H

- ⦿ The base address of the code segment is 11110 H.
- ⦿ Effective address of memory is given by $11110H + 1232H = 12342H$.

2. DS = 3333 H

- ⦿ The base address of the data segment is 33330 H.
- ⦿ Effective address of memory is given by $33330H + 0020H = 33350H$.

3. SS = 2526 H

- ⦿ The base address of the stack segment is 25260 H.
- ⦿ Effective address of memory is given by $25260H + 1100H = 26350H$.